

Módulo 2 — Trabajar con el Modelo

Fundamentos de IA Productiva

Perfil del lector: ingeniero informático con sólida experiencia técnica para quien el paradigma de inteligencia artificial es nuevo. Este documento usa analogías del día a día cuando son más claras que las técnicas, y recurre a conceptos de ingeniería solo cuando simplifican genuinamente.

Formato de la clase: una primera hora de **conceptos y demostración** (secciones 1 a 5) y una segunda hora de **práctica y retroalimentación** (sección 7). La sección 7 está pensada para hacerse en vivo, con el modelo abierto al lado.

Tabla de Contenidos

1. La ventana de contexto - 1.1 Qué es: la mesa de trabajo del modelo - 1.2 Qué ocupa la ventana (todo cuenta) - 1.3 Por qué importa: costo, límite y latencia - 1.4 Context rot: más contexto no es mejor contexto - 1.5 Estrategias para gestionar la ventana
 2. Prompt engineering: la base - 2.1 Claridad y dirección: la regla de oro - 2.2 Dar contexto y motivación (el "por qué") - 2.3 Rol y system prompt - 2.4 Controlar el formato de salida - 2.5 Anatomía de un buen prompt
 3. Estrategias de estructuración - 3.1 Few-shot: enseñar con ejemplos - 3.2 Chain-of-thought: pedir que razone en pasos - 3.3 XML tagging: estructurar el prompt - 3.4 Combinarlas: el patrón completo
 4. Una tarea vs. varias: cuándo dividir - 4.1 Señales de que un prompt hace demasiado - 4.2 Prompt chaining: encadenar pasos - 4.3 Cuándo conviene todo en un solo prompt
 5. Evaluar si una respuesta es buena - 5.1 "Suena bien" no es "es correcto" - 5.2 Los cinco criterios - 5.3 Rúbricas y el modelo como juez - 5.4 Checklist práctico de revisión
 6. Glosario del Módulo
 7. Práctica guiada (segunda hora)
 8. Preguntas de repaso
 9. Recursos extra
-

1. La ventana de contexto

1.1 Qué es: la mesa de trabajo del modelo

La **ventana de contexto** es la cantidad máxima de tokens (ver Módulo 1) que el modelo puede tener "a la vista" en una sola interacción. Incluye todo lo que entra y todo lo que sale: tus instrucciones, el historial de la conversación, los documentos que pegaste y la propia respuesta que está generando.

Analogía · la mesa de trabajo. Imagina un carpintero con una mesa de tamaño fijo. Puede tener encima los planos, las herramientas y la pieza en la que trabaja — pero solo hasta donde quepa. Si amontona demasiado, las cosas del fondo quedan tapadas y empieza a perder de vista lo importante. El modelo es igual: la ventana es esa mesa, y todo lo que necesita "ver" para responder tiene que caber encima a la vez.

Un detalle crucial que choca con la intuición de un ingeniero: el modelo **no tiene memoria entre llamadas**. Cada vez que envías un mensaje, se le reenvía toda la conversación desde el inicio. Lo que parece "memoria" en un chat es, en realidad, el historial completo viajando una y otra vez dentro de la ventana. Cuando la conversación ya no cabe, lo más antiguo se descarta o se resume.

En resumen: la ventana de contexto es la memoria de trabajo del modelo para esta interacción. No es un disco duro ni una base de datos: es un espacio finito que se llena, y lo que no está dentro, para el modelo no existe.

1.2 Qué ocupa la ventana (todo cuenta)

Es fácil pensar que la ventana solo la ocupa "lo que escribo". En realidad la consumen varias fuentes simultáneas:

VENTANA DE CONTEXTO (p. ej. 200k tokens)	
[System prompt]	instrucciones de la plataforma + rol + reglas
[Herramientas]	definiciones de tools disponibles (búsqueda, código...)
[Documentos]	archivos pegados, PDFs, código adjunto
[Historial]	todos los turnos previos de la conversación
[Tu mensaje]	la pregunta o instrucción actual
[Respuesta]	lo que el modelo está generando ahora (también ocupa)

Esto explica comportamientos que de otro modo parecen mágicos o caprichosos:

- En una conversación larga, el modelo "olvida" algo que dijiste al principio: ese tramo se quedó fuera de la ventana o se resumió.
- Pegar un PDF de 80 páginas "consume" gran parte de la mesa antes de que escribas tu pregunta.
- Una respuesta muy larga puede cortarse: no quedaba espacio de salida.

Punto clave para un técnico: la ventana se mide en tokens, no en "mensajes". Un mensaje corto con un archivo adjunto enorme puede ocupar más que cincuenta mensajes de texto. Razonar en tokens, no en turnos, es el cambio mental que hay que hacer.

1.3 Por qué importa: costo, límite y latencia

La gestión de la ventana no es un tecnicismo: impacta directamente tres cosas que te importan en el trabajo real.

DIMENSIÓN	CÓMO LA AFECTA LA VENTANA
Costo	Se cobra por token de entrada y de salida. Reenviar un historial gigante en cada turno multiplica el costo, aunque tu pregunta sea de una línea.
Límite	Si lo que quieres meter no cabe, simplemente no entra. Hay que elegir qué incluir.
Latencia	Más tokens de entrada → más lento. Una ventana medio llena responde más despacio que una limpia.
Calidad	Contrainuitivo pero medido: más contexto puede empeorar la respuesta (ver 1.4).

Caching. Las plataformas modernas permiten *cachear* partes estables del contexto (p. ej. un documento largo o un system prompt extenso) para no pagarlo entero en cada turno. No necesitas configurarlo en una interfaz de chat, pero conviene saber que existe: es la razón por la que reutilizar el mismo documento en una conversación es más barato que pegarlo de nuevo cada vez.

1.4 Context rot: más contexto no es mejor contexto

Uno de los hallazgos más importantes y menos intuitivos de los últimos años: **el rendimiento del modelo se degrada a medida que crece la entrada**, aunque el dato relevante siga estando ahí. Se le llama *context rot* ("pudrición del contexto").

Analogía · el correo enterrado. Tienes un dato importante en un correo. Si está en los últimos tres mensajes de tu bandeja, lo encuentras al instante. Si está enterrado en la posición 4.000 de una bandeja con 8.000 correos, técnicamente "lo tienes" — pero en la práctica casi nunca lo recuperas. El modelo sufre exactamente eso.

Dos fenómenos relacionados:

"Lost in the middle" (perdido en el medio). Los modelos prestan más atención al **inicio** y al **final** del contexto que al medio. La precisión dibuja una curva en forma de U: alta en los extremos, baja en el centro. Cuando el dato relevante se mueve del borde al medio, el rendimiento puede caer **más de un 30%**.

Degradación por longitud. Un estudio de Chroma (2025) probó 18 modelos de frontera (GPT-4.1, Claude 4, Gemini 2.5, Qwen3) y **todos** rindieron peor a medida que crecía la entrada — la caída más pronunciada en el rango de 100k–500k tokens. No es un defecto de un modelo concreto: es una propiedad general de la arquitectura.

Precisión al recuperar un dato según su posición en un contexto largo:



Implicación práctica directa: 1. **Pon lo importante al principio o al final**, no enterrado en el medio. 2. Con documentos largos, **pon el documento arriba y tu pregunta al final** — en pruebas de Anthropic esto mejora la calidad hasta un 30% en entradas complejas. 3. **Menos es más.** Un contexto curado y relevante supera a uno enorme y ruidoso. La habilidad no es "meter todo", es *elegir qué meter*.

1.5 Estrategias para gestionar la ventana

ESTRATEGIA	CUÁNDO USARLA
Curar, no acumular	Siempre. Incluye solo los documentos y el historial que de verdad hacen falta para la tarea actual.
Documento arriba, pregunta abajo	Al trabajar sobre un texto largo. Mejora la precisión y reduce el "lost in the middle".
Pedir citas primero	En análisis de documentos: pídele que primero extraiga las citas relevantes y luego responda. Eso lo ancla al texto y reduce alucinaciones.
Resumir y reiniciar	Cuando una conversación se vuelve muy larga, pídele un resumen del estado y empieza un chat nuevo con ese resumen. Limpia la mesa.
Una tarea por conversación	Mezclar tres temas distintos en un mismo chat ensucia el contexto. Un hilo por objetivo.

Regla mental: trata la ventana como un recurso escaso que tú administras, no como un cajón infinito donde tirar todo. La calidad de la salida depende tanto de lo que *quitas* como de lo que pones.

2. Prompt engineering: la base

El *prompt engineering* es el oficio de escribir instrucciones que produzcan buenas respuestas de forma fiable. No es magia ni "palabras secretas": es comunicación clara aplicada a un colaborador muy capaz pero literal.

El marco mental que más ayuda: trata al modelo como **un empleado nuevo, brillante pero sin contexto** sobre tu empresa, tus convenciones ni lo que tienes en la cabeza. Es muy capaz, pero no adivina. Cuanto más preciso seas sobre lo que quieres, mejor será el resultado.

2.1 Claridad y dirección: la regla de oro

La causa número uno de respuestas malas no es el modelo: es un prompt ambiguo. El modelo responde excepcionalmente bien a instrucciones explícitas y específicas.

La regla de oro (Anthropic). Muéstrale tu prompt a un colega con poco contexto de la tarea y pídele que lo siga. **Si se confunde, el modelo también se confundirá.**

Menos efectivo:

"Crea un dashboard de analítica."

Más efectivo:

"Crea un dashboard de analítica. Incluye tantas funciones e interacciones relevantes como sea posible. Ve más allá de lo básico para lograr una implementación completa."

Dos hábitos concretos:

- **Sé específico sobre el formato y las restricciones** de la salida que esperas.
- **Da instrucciones como pasos secuenciales** (lista numerada) cuando el orden o la completitud importan.

Matiz importante (2026): los modelos actuales siguen instrucciones de forma muy literal y son muy proactivos. El truco de gritar "**CRÍTICO: DEBES hacer X**" que funcionaba en modelos antiguos hoy provoca el efecto contrario (*overtriggering*). Con los modelos actuales basta un lenguaje normal: "*Usa esta herramienta cuando...*".

2.2 Dar contexto y motivación (el "por qué")

Explicarle al modelo *por qué* quieres algo mejora la respuesta, porque puede generalizar a partir de la intención en vez de seguir la regla al pie de la letra.

Menos efectivo:

"NUNCA uses puntos suspensivos."

Más efectivo:

"Tu respuesta la leerá en voz alta un motor de texto-a-voz, así que nunca uses puntos suspensivos porque el motor no sabe pronunciarlos."

Con el "por qué", el modelo entiende el objetivo real y toma decisiones coherentes en casos que tu regla no había previsto. Es la diferencia entre dar una orden y dar contexto a alguien inteligente.

2.3 Rol y system prompt

Asignar un **rol** enfoca el tono y el comportamiento del modelo. Incluso una sola frase hace diferencia. En interfaces de chat, esto suele ir en las "instrucciones personalizadas" o en la descripción de un Proyecto; vía API es el *system prompt*.

Sin rol:

"¿Cómo ordeno una lista de diccionarios por una clave?"

Con rol (system prompt):

"Eres un asistente experto en Python que prioriza código idiomático y legible.
Explicas el porqué de cada decisión en una frase."

Diferencia clave: el **system prompt** define *quién* es el modelo y *cómo* debe comportarse de forma persistente en toda la conversación. El **mensaje de usuario** es la tarea concreta de este turno. Separar ambos mantiene el comportamiento estable entre preguntas.

2.4 Controlar el formato de salida

Tres técnicas que funcionan especialmente bien para dirigir el formato:

1. Di qué hacer, no qué no hacer.

En vez de: "No uses markdown."

Prueba: "Escribe la respuesta en párrafos de prosa que fluyan con naturalidad."

2. Usa indicadores de formato con etiquetas.

"Escribe las secciones de prosa de tu respuesta dentro de etiquetas
<prosa> ... </prosa>."

3. Haz que el estilo de tu prompt se parezca al de la salida deseada. El formato que usas en el prompt influye en el formato de la respuesta. Si escribes el prompt sin markdown, tiendes a recibir menos markdown. Si quieres prosa, escribe en prosa.

Nota de 2026: los modelos actuales son más concisos por defecto y pueden saltarse resúmenes. Si quieres más detalle o un resumen final, **pídelo explícitamente**.

2.5 Anatomía de un buen prompt

No todos los prompts necesitan todas las piezas, pero este es el esqueleto completo, en un orden que funciona bien:

1. ROL / CONTEXTO → quién eres y para qué sirve la tarea
2. INSTRUCCIÓN → qué quieres, en pasos si hace falta
3. CONTEXTO/DATOS → documentos o información (¡arriba si son largos!)
4. EJEMPLOS → 1-5 ejemplos del formato deseado (sección 3.1)
5. FORMATO DE SALIDA → cómo quieres exactamente la respuesta
6. LA PREGUNTA → al final, sobre todo con documentos largos

En resumen: un buen prompt no es largo ni rebuscado. Es **claro, específico, estructurado y honesto sobre lo que quieres**. Casi todos los problemas de calidad se resuelven mejorando el prompt antes que cambiando de modelo.

3. Estrategias de estructuración

Tres técnicas elevan la fiabilidad de las respuestas de forma medible. Se pueden usar solas o combinadas.

3.1 Few-shot: enseñar con ejemplos

El **few-shot prompting** (también *multishot*) consiste en incluir un puñado de ejemplos de entrada → salida dentro del prompt. Es una de las formas más fiables de dirigir el formato, el tono y la estructura.

Analogía · mostrar en vez de explicar. Es la diferencia entre describirle a alguien cómo doblar una camiseta con palabras, y doblar dos camisetas delante de él. Con dos demostraciones, capta el patrón al instante. El modelo aprende igual: ve el patrón y lo replica.

Clasifica el sentimiento de cada reseña como POSITIVO, NEUTRO o NEGATIVO.

<ejemplos>

<ejemplo>Reseña: "Llegó tarde pero el producto está bien." → NEUTRO</ejemplo>

<ejemplo>Reseña: "Una maravilla, lo recomiendo a todos." → POSITIVO</ejemplo>

<ejemplo>Reseña: "Se rompió a la semana, pésima calidad." → NEGATIVO</ejemplo>

</ejemplos>

Reseña: "Cumple lo que promete, sin más." →

Para que los ejemplos funcionen, deben ser:

- **Relevantes:** que reflejen de cerca tu caso real.

- **Diversos:** que cubran casos límite y varíen lo suficiente para que el modelo no agarre un patrón no deseado (p. ej., que todos los ejemplos positivos sean largos).
- **Estructurados:** envueltos en etiquetas `<ejemplo>` (varios en `<ejemplos>`) para que el modelo los distinga de las instrucciones.

Recomendación: entre **3 y 5 ejemplos** suele ser el punto óptimo. Few-shot brilla en tareas de **clasificación, extracción y formato estricto** (JSON, YAML, plantillas).

3.2 Chain-of-thought: pedir que razone en pasos

El **chain-of-thought** (CoT, "cadena de pensamiento") consiste en pedirle al modelo que muestre su razonamiento paso a paso *antes* de dar la respuesta final. Recuerda del Módulo 1 que el modelo construye texto token a token: darle espacio para "pensar en voz alta" mejora notablemente los resultados en tareas que requieren razonamiento.

Analogía · mostrar el desarrollo. Es la diferencia entre exigirle a un estudiante solo el resultado de un problema, y pedirle que muestre el desarrollo. Al desarrollar, comete menos errores — y tú puedes ver dónde se equivocó si falla.

Forma simple (zero-shot CoT):

```
"...resuelve el problema. Piensa paso a paso antes de dar la respuesta final."
```

Forma estructurada (separando razonamiento y respuesta):

```
"Razona dentro de <razonamiento> </razonamiento> y luego da la respuesta final dentro de <respuesta> </respuesta>."
```

La forma estructurada tiene una ventaja práctica: puedes **leer el razonamiento** para detectar dónde falló, y **extraer solo la respuesta** de forma limpia.

Matices importantes: - Para razonamiento, el **zero-shot CoT** ("piensa paso a paso") a menudo supera al few-shot, porque el modelo genera su propia lógica sin quedar atado a ejemplos que quizá no representan bien tu caso. - Los modelos actuales tienen un **modo de "pensamiento" (thinking) adaptativo** que hace esto internamente cuando la tarea lo amerita. En interfaces modernas no siempre hace falta pedir CoT a mano; sí ayuda saber que existe y por qué funciona. - Pídele que **se autoverifique** al final: *"Antes de terminar, comprueba tu respuesta contra [criterio]."* Esto atrapa errores de forma fiable, sobre todo en código y matemática.

3.3 XML tagging: estructurar el prompt

Las **etiquetas XML** (`<instrucciones>` , `<contexto>` , `<ejemplo>` , `<documento>`) ayudan al modelo a separar sin ambigüedad las distintas partes de un prompt complejo: qué son instrucciones, qué son datos, qué son ejemplos. Los modelos Claude en particular fueron entrenados para prestar atención especial a esta estructura.

Analogía · las carpetas rotuladas. Entregarle a alguien un montón de hojas sueltas mezcladas vs. entregarle carpetas rotuladas: "instrucciones", "datos", "ejemplos". Con las carpetas, nadie confunde un dato de entrada con una orden. Las etiquetas XML son esos rótulos.

```
<instrucciones>
  Resume el informe en 3 viñetas para un comité ejecutivo.
</instrucciones>

<documento>
  {{aquí va el texto largo del informe}}
</documento>

<formato>
  Tres viñetas, máximo 20 palabras cada una, sin jerga técnica.
</formato>
```

Buenas prácticas:

- Usa nombres de etiqueta **consistentes y descriptivos** en todos tus prompts.
- **Anida** cuando haya jerarquía natural (varios documentos dentro de `<documentos>` , cada uno en `<documento index="n">`).
- No hay etiquetas "oficiales" obligatorias: lo que importa es que sean claras y que las uses de forma coherente.

3.4 Combinarlas: el patrón completo

Las tres técnicas se potencian juntas. Un prompt robusto típico combina XML para estructurar, few-shot para fijar el formato y CoT para el razonamiento:

```
<instrucciones>
  Eres un analista de soporte. Clasifica cada ticket por urgencia (ALTA/MEDIA/BAJA)
  y justifica brevemente.
</instrucciones>
```

```
<ejemplos>
  <ejemplo>
    Ticket: "No puedo entrar y tengo una demo con un cliente en 10 minutos."
    <razonamiento>Bloqueo total + límite de tiempo inmediato.</razonamiento>
    <respuesta>ALTA</respuesta>
  </ejemplo>
</ejemplos>
```

```
<ticket>
  "Sería bueno poder exportar a Excel algún día."
</ticket>
```

Razona dentro de <razonamiento> y da el veredicto en <respuesta>.

En resumen: XML organiza, few-shot muestra el patrón, CoT da espacio para razonar. Empieza simple; añade estructura solo cuando la respuesta no salga bien a la primera.

4. Una tarea vs. varias: cuándo dividir

Una de las decisiones más prácticas del día a día: ¿meto todo en un solo prompt, o lo parto en pasos?

4.1 Señales de que un prompt hace demasiado

Recuerda del Módulo 1: cuantas más restricciones simultáneas impones, más probable es que el modelo cumpla unas y "olvide" otras. Señales de que un prompt está sobrecargado:

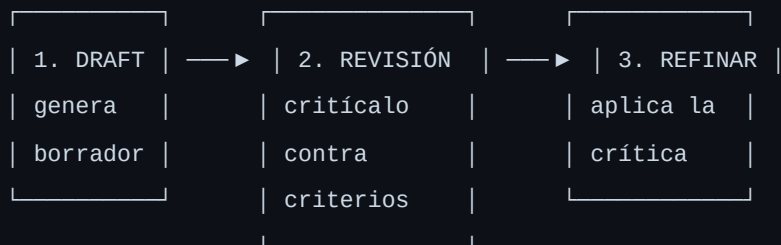
- Pide **varias cosas de naturaleza distinta** a la vez (investigar **y** redactar **y** formatear **y** traducir).
- El resultado cumple unas instrucciones e ignora otras, de forma inconsistente.
- Necesitas **inspeccionar un paso intermedio** antes de seguir.
- El prompt se ha vuelto tan largo que ya no sabes qué parte está fallando.

Analogía · el pedido en el restaurante. Puedes pedirle al camarero "tráeme la carta, recomiéndame un plato, sírvenme el vino y trae la cuenta" todo de golpe — pero algo se perderá. Pedir paso a paso, esperando cada respuesta, sale mejor. Con tareas complejas, el modelo agradece lo mismo.

4.2 Prompt chaining: encadenar pasos

El **prompt chaining** descompone una tarea en una secuencia de pasos, donde la salida de uno es la entrada del siguiente. Cada paso es más simple, más fiable y más fácil de depurar, porque el modelo se concentra en una sola cosa a la vez.

El patrón más común y útil es la **autocorrección**:



- **Paso 1 — Draft:** "Escribe un primer borrador del correo a clientes."
- **Paso 2 — Revisión:** "Revisa este borrador. ¿Es claro? ¿Tono adecuado? Lista 3 mejoras concretas."
- **Paso 3 — Refinar:** "Reescribe el correo aplicando esas mejoras."

Otras formas de descomposición útiles:

PATRÓN	IDEA	EJEMPLO
Encadenado (chaining)	Pasos fijos en secuencia	Extraer → resumir → traducir
Paralelo	Subtareas independientes a la vez	Analizar 5 documentos por separado y luego unir
Borrador → revisión → refinamiento	Autocorrección	(el ejemplo de arriba)

Ventaja oculta del chaining: puedes **ver y corregir cada paso**. Si el resumen salió mal, lo arreglas ahí, sin rehacer toda la cadena. Es depuración aplicada a prompts. Este flujo iterativo es el corazón del Módulo 4.

4.3 Cuándo conviene todo en un solo prompt

Dividir no siempre es mejor: cada paso extra cuesta tokens, latencia y complejidad. Conviene un solo prompt cuando:

- La tarea es **cohesiva** y cabe sin sobrecargar (resumir un texto, responder una pregunta, reformatear datos).
- No necesitas inspeccionar pasos intermedios.
- El modelo ya la resuelve bien a la primera.

SITUACIÓN	MEJOR ENFOQUE
Resumir un documento	Un solo prompt
Extraer datos a JSON	Un solo prompt (con few-shot)
Investigar, redactar y maquetar un informe	Encadenar pasos
Tarea donde necesitas validar un paso intermedio	Encadenar pasos
Procesar 50 elementos independientes	Paralelo (uno por uno o por lotes)
Razonamiento complejo de varios pasos	Un prompt con CoT / thinking

Regla práctica: empieza con **un solo prompt bien escrito**. Solo divide cuando veas que el modelo se atraganta, cuando necesites inspeccionar un paso, o cuando una parte falle de forma repetida. No optimices la complejidad antes de necesitarla.

5. Evaluar si una respuesta es buena

Generar es la mitad fácil. La habilidad que separa a un usuario novato de uno experto es **saber juzgar la respuesta** — no solo si suena bien.

5.1 "Suena bien" no es "es correcto"

Los modelos producen texto fluido, seguro y bien estructurado **incluso cuando se equivocan**. Esa fluidez es justamente la trampa: nuestro cerebro asocia confianza y buena redacción con veracidad, y el modelo es excelente en ambas cosas por diseño (es lo que vimos sobre alucinaciones y RLHF en el Módulo 1).

Analogía · el candidato elocuente. En una entrevista, un candidato que habla con seguridad, estructura bien sus frases y nunca duda, *parece* el mejor. Pero la elocuencia no es competencia. Hay que verificar lo que dice, no cómo lo dice. Con el modelo, igual.

Cuidado con la adulación (*sycophancy*). Los modelos tienden a estar de acuerdo contigo y a reforzar lo que pareces querer oír. Si preguntas "¿verdad que esto está bien?", es más probable que diga que sí. Pregunta en neutro: "*Encuentra los problemas de esto*" en vez de "*¿está bien esto?*".

5.2 Los cinco criterios

En lugar de un juicio global ("me gusta / no me gusta"), evalúa por criterios concretos. Cinco bastan para la mayoría de los casos:

CRITERIO	PREGUNTA DE CONTROL
Correctitud	¿Los hechos, datos y afirmaciones son verdaderos y verificables?
Compleitud	¿Cubre todo lo que pedí, o se dejó partes en el tintero?
Coherencia	¿El razonamiento se sostiene? ¿Hay saltos lógicos o contradicciones?
Fidelidad a la fuente	Si le di un documento, ¿la respuesta se basa en él o se lo inventó?
Formato y utilidad	¿Cumple el formato pedido? ¿Me sirve tal cual o tengo que rehacerlo?

Por qué criterio a criterio (no global): un puntaje global ("7/10") esconde *qué* falló. Evaluar por criterios te dice exactamente dónde está el problema — y si es de correctitud, vas a verificar; si es de formato, ajustas el prompt.

5.3 Rúbricas y el modelo como juez

Cuando evalúas muchas respuestas (o quieres consistencia), formaliza los criterios en una **rúbrica**: una lista explícita de qué hace que una respuesta sea buena.

- **Rúbrica holística:** un solo puntaje global. Rápida pero opaca: no dices *por qué*.
- **Rúbrica analítica:** un puntaje por criterio. Más trabajo, pero te muestra exactamente qué capacidad mejoró o empeoró entre versiones. **Es la recomendada.**

El modelo como juez (*LLM-as-judge*). Puedes usar un modelo para evaluar la salida de otro (o de sí mismo) contra una rúbrica. Es muy útil para revisar a escala, con dos cautelas:

- Dale **criterios específicos**, no "¿está bien?". Pídele puntaje **por criterio**.

- Tiene sesgos (tiende a premiar respuestas largas y bien formateadas). Úsalo como primer filtro, no como veredicto final en lo que importa.

Eres un evaluador estricto. Califica la RESPUESTA contra estos criterios, de 1 a 5 cada uno, con una frase de justificación:

- Correctitud - Completitud - Coherencia - Fidelidad a la fuente

<fuente>{{documento original}}</fuente>

<respuesta>{{la respuesta a evaluar}}</respuesta>

5.4 Checklist práctico de revisión

Una rutina de 30 segundos antes de dar por buena una respuesta:

- ☐ **¿Hay datos específicos** (fechas, cifras, nombres, citas)? → Verifícalos por fuera.
- ☐ **¿Le di una fuente?** → ¿La respuesta se sostiene en ella o inventó cosas?
- ☐ **¿Pedí varias cosas?** → ¿Cumplió todas, o se saltó alguna?
- ☐ **¿El razonamiento se sostiene** leído paso a paso, no solo la conclusión?
- ☐ **¿Estoy preguntando en neutro**, o le di pistas de lo que quería oír?
- ☐ **¿La estoy aceptando porque es correcta, o porque suena bien y estoy cansado?**

Principio clave (enlaza con el Módulo 1): el modelo es un colaborador capaz, no un oráculo. Tu valor no está en generar — el modelo genera más rápido que tú — sino en **dirigir y verificar**. La evaluación es donde pones ese valor.

6. Glosario del Módulo

TÉRMINO	DEFINICIÓN BREVE
Ventana de contexto	Cantidad máxima de tokens que el modelo procesa de una vez; incluye instrucciones, historial, documentos y respuesta
System prompt	Instrucción persistente que define el rol y comportamiento del modelo durante toda la conversación
Context rot	Degradación de la calidad de la respuesta a medida que crece la cantidad de contexto, aunque el dato relevante siga presente
Lost in the middle	Tendencia del modelo a prestar menos atención a la información ubicada en el centro del contexto que en los extremos
Caching (de contexto)	Reutilizar partes estables del contexto entre llamadas para reducir costo y latencia
Prompt engineering	Oficio de redactar instrucciones que producen buenas respuestas de forma fiable
Few-shot / multishot	Incluir varios ejemplos de entrada → salida en el prompt para fijar formato y estilo
Zero-shot	Pedir la tarea sin ejemplos, solo con la instrucción
Chain-of-thought (CoT)	Pedir al modelo que muestre su razonamiento paso a paso antes de la respuesta final
Thinking (adaptativo)	Capacidad de los modelos actuales de razonar internamente cuando la tarea lo amerita
XML tagging	Usar etiquetas (<code><contexto></code> , <code><ejemplo></code> ...) para separar las partes de un prompt sin ambigüedad
Prompt chaining	Descomponer una tarea en pasos secuenciales, donde la salida de uno alimenta al siguiente
Autocorrección	Patrón de chaining: generar borrador → revisar contra criterios → refinar
Rúbrica	Lista explícita de criterios para evaluar la calidad de una respuesta
LLM-as-judge	Usar un modelo para evaluar la salida de otro modelo contra una rúbrica
Sycophancy	Tendencia del modelo a estar de acuerdo con el usuario y reforzar lo que parece querer oír

7. Práctica guiada (segunda hora)

Estos ejercicios están pensados para hacerse en vivo con el modelo abierto. El objetivo no es "acertar", sino **observar la diferencia** que hacen las técnicas. Tras cada uno, comparte el resultado y recibe retroalimentación.

Ejercicio 1 — Antes y después (claridad) · 10 min

Toma una tarea real tuya (redactar un correo, generar un script, resumir un documento). 1. Escribe el prompt "como lo harías normalmente" y guarda la respuesta. 2. Reescríbelo aplicando 2.1–2.4: rol, instrucción específica, el "por qué" y formato. 3. Compara. **Pregunta de reflexión:** ¿qué cambió y por qué?

Ejercicio 2 — Few-shot en acción · 10 min

Pide una clasificación o extracción (p. ej., clasificar 5 frases por sentimiento). 1. Primero **sin ejemplos**. 2. Luego con **3 ejemplos** bien estructurados en etiquetas. 3. Observa la consistencia del formato entre ambos.

Ejercicio 3 — El poder de "piensa paso a paso" · 10 min

Dale un problema de razonamiento con varios pasos (un acertijo lógico o un cálculo con condiciones). 1. Pídele **solo la respuesta**. 2. Pídele que **razone en** `<razonamiento>` **y responda en** `<respuesta>`. 3. ¿Cambió la respuesta? ¿Puedes ver dónde se equivocaría si fallara?

Ejercicio 4 — Encadenar: borrador → crítica → refinamiento · 15 min

Elige algo a redactar (un mensaje difícil, una descripción de producto). 1. Genera el borrador. 2. En el **mismo chat**, pide: "Critica este borrador y lista 3 mejoras concretas." 3. Pide: "Reescríbelo aplicando esas mejoras." 4. **Reflexión:** ¿el resultado encadenado es mejor que pedirlo todo de una vez?

Ejercicio 5 — Caza de errores (evaluación) · 15 min

Pídele un dato verificable sobre un tema de nicho (una fecha, una cifra, una cita). 1. Aplica el **checklist 5.4**: ¿es específico? ¿lo puedes verificar? 2. Verifica el dato por fuera. 3. Vuelve a preguntar **en neutro**: "Encuentra los problemas o imprecisiones de tu respuesta anterior." Observa si se autocorrigió o se adula.

Cierre de la práctica: ¿qué técnica vas a incorporar a tu trabajo desde mañana?

8. Preguntas de repaso

Estas preguntas consolidan los conceptos antes de pasar al Módulo 3. No buscan una respuesta "de examen": el objetivo es razonar en voz alta.

1. Tienes un PDF de 100 páginas y una pregunta concreta sobre algo que está en la página
2. ¿Cómo estructurarías el prompt para maximizar la probabilidad de una buena respuesta, sabiendo lo que sabes sobre *context rot* y *lost in the middle*?
3. Un compañero dice: "uso siempre la ventana de contexto más grande posible para que el modelo tenga toda la información". ¿Qué le responderías?
4. ¿Cuándo usarías **few-shot** y cuándo **chain-of-thought**? Da un ejemplo de tarea para cada uno donde la otra técnica sería peor opción.
5. Te piden automatizar la redacción de respuestas a reseñas de clientes. ¿Lo harías en un solo prompt o encadenado? Justifica con los criterios de la sección 4.
6. El modelo te entrega una respuesta impecablemente redactada y segura. Enumera tres cosas que harías antes de confiar en ella para un entregable importante.
7. Explícale a alguien sin contexto técnico por qué "el modelo no recuerda" lo que le dijiste hace media hora en otro chat, aunque "parezca" que tiene memoria.

9. Recursos extra

Recursos seleccionados para profundizar. Los oficiales de Anthropic son la referencia primaria; el resto aporta evidencia o perspectivas complementarias.

Documentación oficial (referencia primaria) - [Prompting best practices — Claude Docs](#) — la guía canónica: claridad, ejemplos, XML, thinking, contexto largo. - [Building Effective AI Agents — Anthropic](#) — patrones de descomposición: prompt chaining, paralelización, orquestador-trabajadores. - [Use examples \(multishot prompting\) — Claude Docs](#) — cómo elegir y estructurar ejemplos.

Ventana de contexto y context rot - [Context Rot: How Increasing Input Tokens Impacts LLM Performance — Chroma \(2025\)](#) — estudio de 18 modelos; la evidencia clave de que más contexto degrada el rendimiento. - ["Lost in the Middle: How Language Models Use Long Contexts" — Liu et al. \(paper original\)](#) — el trabajo fundacional sobre la curva en U. - [Context Rot — guía explicativa \(Morph\)](#) — resumen accesible del fenómeno y mitigaciones.

Estructuración: few-shot, CoT, XML - [Zero-Shot vs Few-Shot prompting: guía con ejemplos — Vellum](#) - [Chain of Thought Prompting \(CoT\): todo lo que necesitas saber — Vellum](#) - [Prompt engineering across OpenAI, Anthropic & Gemini — Steve Kinney](#) — comparativa práctica entre proveedores.

Anterior: [Módulo 1 — Cómo piensan los modelos](#) Siguiente: [Módulo 3 — Interfaz y ecosistema](#)

Versión 1.0 — Módulo 2 de 7 | Curso: Fundamentos de IA Productiva Actualizado: junio 2026